

```

1 package com.codes.java;
2
3 import com.codes.java.codingquestions.arrays.MajorityElement;
4
5 import java.util.*;
6 import java.util.concurrent.BlockingQueue;
7 import java.util.concurrent.LinkedBlockingQueue;
8 import java.util.stream.Collectors;
9 import java.util.stream.IntStream;
10
11 public class ArrayCodingQuestionsCode {
12
13
14     // =====
15     =
16     // Question: Find Largest Element
17     // Input : {10, 20, 30, 40, 50}
18     // Output: 50
19     // Time Complexity : O(n)
20     // Space Complexity: O(1)
21
22     // =====
23     =
24     public static int findLargest(int[] arr) {
25
26         if (arr == null || arr.length == 0)
27             throw new IllegalArgumentException("Array is empty");
28
29         int largest = arr[0];
30
31         for (int num : arr) {
32             if (num > largest) {
33                 largest = num;
34             }
35         }
36
37         return largest;
38     }
39
40
41     // =====
42     =
43     // Question: Find Smallest Element
44     // Input : {10, 20, 30, 40, 50}
45     // Output: 10
46     // Time Complexity : O(n)
47     // Space Complexity: O(1)
48
49     // =====
50     =
51     public static int findSmallest(int[] arr) {
52
53         if (arr == null || arr.length == 0)
54             throw new IllegalArgumentException("Array is empty");
55
56         int smallest = arr[0];
57
58         for (int num : arr) {
59             if (num < smallest) {
60                 smallest = num;
61             }
62         }
63
64         return smallest;
65     }
66
67
68     // =====
69     =
70     // Question: Find Second Largest Element
71     // Input : {10, 20, 30, 40, 50}
72     // Output: 40
73     // Time Complexity : O(n)
74     // Space Complexity: O(1)
75
76     // =====
77     =
78     public static int findSecondLargest(int[] arr) {
79
80         if (arr == null || arr.length == 0)
81             throw new IllegalArgumentException("Array is empty");
82
83         int largest = arr[0];
84         int secondLargest = -1;
85
86         for (int num : arr) {
87             if (num > largest) {
88                 secondLargest = largest;
89                 largest = num;
90             } else if (num > secondLargest && num < largest) {
91                 secondLargest = num;
92             }
93         }
94
95         return secondLargest;
96     }
97
98
99     // =====
100    =
101    // Question: Find Second Smallest Element
102    // Input : {10, 20, 30, 40, 50}
103    // Output: 20
104    // Time Complexity : O(n)
105    // Space Complexity: O(1)
106
107    // =====
108    =
109    public static int findSecondSmallest(int[] arr) {
110
111        if (arr == null || arr.length == 0)
112            throw new IllegalArgumentException("Array is empty");
113
114        int smallest = arr[0];
115        int secondSmallest = -1;
116
117        for (int num : arr) {
118            if (num < smallest) {
119                secondSmallest = smallest;
120                smallest = num;
121            } else if (num < secondSmallest && num > smallest) {
122                secondSmallest = num;
123            }
124        }
125
126        return secondSmallest;
127    }
128
129
130    // =====
131    =
132    // Question: Find Kth Largest Element
133    // Input : {10, 20, 30, 40, 50}
134    // Output: 30
135    // Time Complexity : O(n)
136    // Space Complexity: O(1)
137
138    // =====
139    =
140    public static int findKthLargest(int[] arr, int k) {
141
142        if (arr == null || arr.length == 0)
143            throw new IllegalArgumentException("Array is empty");
144
145        int largest = arr[0];
146        int secondLargest = -1;
147
148        for (int num : arr) {
149            if (num > largest) {
150                secondLargest = largest;
151                largest = num;
152            } else if (num > secondLargest && num < largest) {
153                secondLargest = num;
154            }
155        }
156
157        return secondLargest;
158    }
159
160
161    // =====
162    =
163    // Question: Find Kth Smallest Element
164    // Input : {10, 20, 30, 40, 50}
165    // Output: 30
166    // Time Complexity : O(n)
167    // Space Complexity: O(1)
168
169    // =====
170    =
171    public static int findKthSmallest(int[] arr, int k) {
172
173        if (arr == null || arr.length == 0)
174            throw new IllegalArgumentException("Array is empty");
175
176        int smallest = arr[0];
177        int secondSmallest = -1;
178
179        for (int num : arr) {
180            if (num < smallest) {
181                secondSmallest = smallest;
182                smallest = num;
183            } else if (num < secondSmallest && num > smallest) {
184                secondSmallest = num;
185            }
186        }
187
188        return secondSmallest;
189    }
190
191
192    // =====
193    =
194    // Question: Find Majority Element
195    // Input : {10, 20, 30, 40, 50}
196    // Output: 50
197    // Time Complexity : O(n)
198    // Space Complexity: O(1)
199
200    // =====
201    =
202    public static int findMajorityElement(int[] arr) {
203
204        if (arr == null || arr.length == 0)
205            throw new IllegalArgumentException("Array is empty");
206
207        int largest = arr[0];
208        int count = 1;
209
210        for (int num : arr) {
211            if (num == largest) {
212                count++;
213            } else {
214                count--;
215            }
216
217            if (count == 0) {
218                largest = num;
219                count = 1;
220            }
221        }
222
223        return largest;
224    }
225
226
227    // =====
228    =
229    // Question: Find Majority Element II
230    // Input : {10, 20, 30, 40, 50}
231    // Output: 50
232    // Time Complexity : O(n)
233    // Space Complexity: O(1)
234
235    // =====
236    =
237    public static int findMajorityElementII(int[] arr) {
238
239        if (arr == null || arr.length == 0)
240            throw new IllegalArgumentException("Array is empty");
241
242        int largest = arr[0];
243        int count = 1;
244
245        for (int num : arr) {
246            if (num == largest) {
247                count++;
248            } else {
249                count--;
250            }
251
252            if (count == 0) {
253                largest = num;
254                count = 1;
255            }
256        }
257
258        return largest;
259    }
260
261
262    // =====
263    =
264    // Question: Find Majority Element III
265    // Input : {10, 20, 30, 40, 50}
266    // Output: 50
267    // Time Complexity : O(n)
268    // Space Complexity: O(1)
269
270    // =====
271    =
272    public static int findMajorityElementIII(int[] arr) {
273
274        if (arr == null || arr.length == 0)
275            throw new IllegalArgumentException("Array is empty");
276
277        int largest = arr[0];
278        int count = 1;
279
280        for (int num : arr) {
281            if (num == largest) {
282                count++;
283            } else {
284                count--;
285            }
286
287            if (count == 0) {
288                largest = num;
289                count = 1;
290            }
291        }
292
293        return largest;
294    }
295
296
297    // =====
298    =
299    // Question: Find Majority Element IV
300    // Input : {10, 20, 30, 40, 50}
301    // Output: 50
302    // Time Complexity : O(n)
303    // Space Complexity: O(1)
304
305    // =====
306    =
307    public static int findMajorityElementIV(int[] arr) {
308
309        if (arr == null || arr.length == 0)
310            throw new IllegalArgumentException("Array is empty");
311
312        int largest = arr[0];
313        int count = 1;
314
315        for (int num : arr) {
316            if (num == largest) {
317                count++;
318            } else {
319                count--;
320            }
321
322            if (count == 0) {
323                largest = num;
324                count = 1;
325            }
326        }
327
328        return largest;
329    }
330
331
332    // =====
333    =
334    // Question: Find Majority Element V
335    // Input : {10, 20, 30, 40, 50}
336    // Output: 50
337    // Time Complexity : O(n)
338    // Space Complexity: O(1)
339
340    // =====
341    =
342    public static int findMajorityElementV(int[] arr) {
343
344        if (arr == null || arr.length == 0)
345            throw new IllegalArgumentException("Array is empty");
346
347        int largest = arr[0];
348        int count = 1;
349
350        for (int num : arr) {
351            if (num == largest) {
352                count++;
353            } else {
354                count--;
355            }
356
357            if (count == 0) {
358                largest = num;
359                count = 1;
360            }
361        }
362
363        return largest;
364    }
365
366
367    // =====
368    =
369    // Question: Find Majority Element VI
370    // Input : {10, 20, 30, 40, 50}
371    // Output: 50
372    // Time Complexity : O(n)
373    // Space Complexity: O(1)
374
375    // =====
376    =
377    public static int findMajorityElementVI(int[] arr) {
378
379        if (arr == null || arr.length == 0)
380            throw new IllegalArgumentException("Array is empty");
381
382        int largest = arr[0];
383        int count = 1;
384
385        for (int num : arr) {
386            if (num == largest) {
387                count++;
388            } else {
389                count--;
390            }
391
392            if (count == 0) {
393                largest = num;
394                count = 1;
395            }
396        }
397
398        return largest;
399    }
400
401
402    // =====
403    =
404    // Question: Find Majority Element VII
405    // Input : {10, 20, 30, 40, 50}
406    // Output: 50
407    // Time Complexity : O(n)
408    // Space Complexity: O(1)
409
410    // =====
411    =
412    public static int findMajorityElementVII(int[] arr) {
413
414        if (arr == null || arr.length == 0)
415            throw new IllegalArgumentException("Array is empty");
416
417        int largest = arr[0];
418        int count = 1;
419
420        for (int num : arr) {
421            if (num == largest) {
422                count++;
423            } else {
424                count--;
425            }
426
427            if (count == 0) {
428                largest = num;
429                count = 1;
430            }
431        }
432
433        return largest;
434    }
435
436
437    // =====
438    =
439    // Question: Find Majority Element VIII
440    // Input : {10, 20, 30, 40, 50}
441    // Output: 50
442    // Time Complexity : O(n)
443    // Space Complexity: O(1)
444
445    // =====
446    =
447    public static int findMajorityElementVIII(int[] arr) {
448
449        if (arr == null || arr.length == 0)
450            throw new IllegalArgumentException("Array is empty");
451
452        int largest = arr[0];
453        int count = 1;
454
455        for (int num : arr) {
456            if (num == largest) {
457                count++;
458            } else {
459                count--;
460            }
461
462            if (count == 0) {
463                largest = num;
464                count = 1;
465            }
466        }
467
468        return largest;
469    }
470
471
472    // =====
473    =
474    // Question: Find Majority Element IX
475    // Input : {10, 20, 30, 40, 50}
476    // Output: 50
477    // Time Complexity : O(n)
478    // Space Complexity: O(1)
479
480    // =====
481    =
482    public static int findMajorityElementIX(int[] arr) {
483
484        if (arr == null || arr.length == 0)
485            throw new IllegalArgumentException("Array is empty");
486
487        int largest = arr[0];
488        int count = 1;
489
490        for (int num : arr) {
491            if (num == largest) {
492                count++;
493            } else {
494                count--;
495            }
496
497            if (count == 0) {
498                largest = num;
499                count = 1;
500            }
501        }
502
503        return largest;
504    }
505
506
507    // =====
508    =
509    // Question: Find Majority Element X
510    // Input : {10, 20, 30, 40, 50}
511    // Output: 50
512    // Time Complexity : O(n)
513    // Space Complexity: O(1)
514
515    // =====
516    =
517    public static int findMajorityElementX(int[] arr) {
518
519        if (arr == null || arr.length == 0)
520            throw new IllegalArgumentException("Array is empty");
521
522        int largest = arr[0];
523        int count = 1;
524
525        for (int num : arr) {
526            if (num == largest) {
527                count++;
528            } else {
529                count--;
530            }
531
532            if (count == 0) {
533                largest = num;
534                count = 1;
535            }
536        }
537
538        return largest;
539    }
540
541
542    // =====
543    =
544    // Question: Find Majority Element XI
545    // Input : {10, 20, 30, 40, 50}
546    // Output: 50
547    // Time Complexity : O(n)
548    // Space Complexity: O(1)
549
550    // =====
551    =
552    public static int findMajorityElementXI(int[] arr) {
553
554        if (arr == null || arr.length == 0)
555            throw new IllegalArgumentException("Array is empty");
556
557        int largest = arr[0];
558        int count = 1;
559
560        for (int num : arr) {
561            if (num == largest) {
562                count++;
563            } else {
564                count--;
565            }
566
567            if (count == 0) {
568                largest = num;
569                count = 1;
570            }
571        }
572
573        return largest;
574    }
575
576
577    // =====
578    =
579    // Question: Find Majority Element XII
580    // Input : {10, 20, 30, 40, 50}
581    // Output: 50
582    // Time Complexity : O(n)
583    // Space Complexity: O(1)
584
585    // =====
586    =
587    public static int findMajorityElementXII(int[] arr) {
588
589        if (arr == null || arr.length == 0)
590            throw new IllegalArgumentException("Array is empty");
591
592        int largest = arr[0];
593        int count = 1;
594
595        for (int num : arr) {
596            if (num == largest) {
597                count++;
598            } else {
599                count--;
600            }
601
602            if (count == 0) {
603                largest = num;
604                count = 1;
605            }
606        }
607
608        return largest;
609    }
610
611
612    // =====
613    =
614    // Question: Find Majority Element XIII
615    // Input : {10, 20, 30, 40, 50}
616    // Output: 50
617    // Time Complexity : O(n)
618    // Space Complexity: O(1)
619
620    // =====
621    =
622    public static int findMajorityElementXIII(int[] arr) {
623
624        if (arr == null || arr.length == 0)
625            throw new IllegalArgumentException("Array is empty");
626
627        int largest = arr[0];
628        int count = 1;
629
630        for (int num : arr) {
631            if (num == largest) {
632                count++;
633            } else {
634                count--;
635            }
636
637            if (count == 0) {
638                largest = num;
639                count = 1;
640            }
641        }
642
643        return largest;
644    }
645
646
647    // =====
648    =
649    // Question: Find Majority Element XIV
650    // Input : {10, 20, 30, 40, 50}
651    // Output: 50
652    // Time Complexity : O(n)
653    // Space Complexity: O(1)
654
655    // =====
656    =
657    public static int findMajorityElementXIV(int[] arr) {
658
659        if (arr == null || arr.length == 0)
660            throw new IllegalArgumentException("Array is empty");
661
662        int largest = arr[0];
663        int count = 1;
664
665        for (int num : arr) {
666            if (num == largest) {
667                count++;
668            } else {
669                count--;
670            }
671
672            if (count == 0) {
673                largest = num;
674                count = 1;
675            }
676        }
677
678        return largest;
679    }
680
681
682    // =====
683    =
684    // Question: Find Majority Element XV
685    // Input : {10, 20, 30, 40, 50}
686    // Output: 50
687    // Time Complexity : O(n)
688    // Space Complexity: O(1)
689
690    // =====
691    =
692    public static int findMajorityElementXV(int[] arr) {
693
694        if (arr == null || arr.length == 0)
695            throw new IllegalArgumentException("Array is empty");
696
697        int largest = arr[0];
698        int count = 1;
699
700        for (int num : arr) {
701            if (num == largest) {
702                count++;
703            } else {
704                count--;
705            }
706
707            if (count == 0) {
708                largest = num;
709                count = 1;
710            }
711        }
712
713        return largest;
714    }
715
716
717    // =====
718    =
719    // Question: Find Majority Element XVI
720    // Input : {10, 20, 30, 40, 50}
721    // Output: 50
722    // Time Complexity : O(n)
723    // Space Complexity: O(1)
724
725    // =====
726    =
727    public static int findMajorityElementXVI(int[] arr) {
728
729        if (arr == null || arr.length == 0)
730            throw new IllegalArgumentException("Array is empty");
731
732        int largest = arr[0];
733        int count = 1;
734
735        for (int num : arr) {
736            if (num == largest) {
737                count++;
738            } else {
739                count--;
740            }
741
742            if (count == 0) {
743                largest = num;
744                count = 1;
745            }
746        }
747
748        return largest;
749    }
750
751
752    // =====
753    =
754    // Question: Find Majority Element XVII
755    // Input : {10, 20, 30, 40, 50}
756    // Output: 50
757    // Time Complexity : O(n)
758    // Space Complexity: O(1)
759
760    // =====
761    =
762    public static int findMajorityElementXVII(int[] arr) {
763
764        if (arr == null || arr.length == 0)
765            throw new IllegalArgumentException("Array is empty");
766
767        int largest = arr[0];
768        int count = 1;
769
770        for (int num : arr) {
771            if (num == largest) {
772                count++;
773            } else {
774                count--;
775            }
776
777            if (count == 0) {
778                largest = num;
779                count = 1;
780            }
781        }
782
783        return largest;
784    }
785
786
787    // =====
788    =
789    // Question: Find Majority Element XVIII
790    // Input : {10, 20, 30, 40, 50}
791    // Output: 50
792    // Time Complexity : O(n)
793    // Space Complexity: O(1)
794
795    // =====
796    =
797    public static int findMajorityElementXVIII(int[] arr) {
798
799        if (arr == null || arr.length == 0)
800            throw new IllegalArgumentException("Array is empty");
801
802        int largest = arr[0];
803        int count = 1;
804
805        for (int num : arr) {
806            if (num == largest) {
807                count++;
808            } else {
809                count--;
810            }
811
812            if (count == 0) {
813                largest = num;
814                count = 1;
815            }
816        }
817
818        return largest;
819    }
820
821
822    // =====
823    =
824    // Question: Find Majority Element XIX
825    // Input : {10, 20, 30, 40, 50}
826    // Output: 50
827    // Time Complexity : O(n)
828    // Space Complexity: O(1)
829
830    // =====
831    =
832    public static int findMajorityElementXIX(int[] arr) {
833
834        if (arr == null || arr.length == 0)
835            throw new IllegalArgumentException("Array is empty");
836
837        int largest = arr[0];
838        int count = 1;
839
840        for (int num : arr) {
841            if (num == largest) {
842                count++;
843            } else {
844                count--;
845            }
846
847            if (count == 0) {
848                largest = num;
849                count = 1;
850            }
851        }
852
853        return largest;
854    }
855
856
857    // =====
858    =
859    // Question: Find Majority Element XX
860    // Input : {10, 20, 30, 40, 50}
861    // Output: 50
862    // Time Complexity : O(n)
863    // Space Complexity: O(1)
864
865    // =====
866    =
867    public static int findMajorityElementXX(int[] arr) {
868
869        if (arr == null || arr.length == 0)
870            throw new IllegalArgumentException("Array is empty");
871
872        int largest = arr[0];
873        int count = 1;
874
875        for (int num : arr) {
876            if (num == largest) {
877                count++;
878            } else {
879                count--;
880            }
881
882            if (count == 0) {
883                largest = num;
884                count = 1;
885            }
886        }
887
888        return largest;
889    }
890
891
892    // =====
893    =
894    // Question: Find Majority Element XXI
895    // Input : {10, 20, 30, 40, 50}
896    // Output: 50
897    // Time Complexity : O(n)
898    // Space Complexity: O(1)
899
900    // =====
901    =
902    public static int findMajorityElementXXI(int[] arr) {
903
904        if (arr == null || arr.length == 0)
905            throw new IllegalArgumentException("Array is empty");
906
907        int largest = arr[0];
908        int count = 1;
909
910        for (int num : arr) {
911            if (num == largest) {
912                count++;
913            } else {
914                count--;
915            }
916
917            if (count == 0) {
918                largest = num;
919                count = 1;
920            }
921        }
922
923        return largest;
924    }
925
926
927    // =====
928    =
929    // Question: Find Majority Element XXII
930    // Input : {10, 20, 30, 40, 50}
931    // Output: 50
932    // Time Complexity : O(n)
933    // Space Complexity: O(1)
934
935    // =====
936    =
937    public static int findMajorityElementXXII(int[] arr) {
938
939        if (arr == null || arr.length == 0)
940            throw new IllegalArgumentException("Array is empty");
941
942        int largest = arr[0];
943        int count = 1;
944
945        for (int num : arr) {
946            if (num == largest) {
947                count++;
948            } else {
949                count--;
950            }
951
952            if (count == 0) {
953                largest = num;
954                count = 1;
955            }
956        }
957
958        return largest;
959    }
960
961
962    // =====
963    =
964    // Question: Find Majority Element XXIII
965    // Input : {10, 20, 30, 40, 50}
966    // Output: 50
967    // Time Complexity : O(n)
968    // Space Complexity: O(1)
969
970    // =====
971    =
972    public static int findMajorityElementXXIII(int[] arr) {
973
974        if (arr == null || arr.length == 0)
975            throw new IllegalArgumentException("Array is empty");
976
977        int largest = arr[0];
978        int count = 1;
979
980        for (int num : arr) {
981            if (num == largest) {
982                count++;
983            } else {
984                count--;
985            }
986
987            if (count == 0) {
988                largest = num;
989                count = 1;
990            }
991        }
992
993        return largest;
994    }
995
996
997    // =====
998    =
999    // Question: Find Majority Element XXIV
1000   // Input : {10, 20, 30, 40, 50}
1001   // Output: 50
1002   // Time Complexity : O(n)
1003   // Space Complexity: O(1)
1004
1005   // =====
1006   =
1007   public static int findMajorityElementXXIV(int[] arr) {
1008
1009       if (arr == null || arr.length == 0)
1010           throw new IllegalArgumentException("Array is empty");
1011
1012       int largest = arr[0];
1013       int count = 1;
1014
1015       for (int num : arr) {
1016           if (num == largest) {
1017               count++;
1018           } else {
1019               count--;
1020           }
1021
1022           if (count == 0) {
1023               largest = num;
1024               count = 1;
1025           }
1026       }
1027
1028       return largest;
1029   }
1030
1031
1032   // =====
1033   =
1034   // Question: Find Majority Element XXV
1035   // Input : {10, 20, 30, 40, 50}
1036   // Output: 50
1037   // Time Complexity : O(n)
1038   // Space Complexity: O(1)
1039
1040   // =====
1041   =
1042   public static int findMajorityElementXXV(int[] arr) {
1043
1044       if (arr == null || arr.length == 0)
1045           throw new IllegalArgumentException("Array is empty");
1046
1047       int largest = arr[0];
1048       int count = 1;
1049
1050       for (int num : arr) {
1051           if (num == largest) {
1052               count++;
1053           } else {
1054               count--;
1055           }
1056
1057           if (count == 0) {
1058               largest = num;
1059               count = 1;
1060           }
1061       }
1062
1063       return largest;
1064   }
1065
1066
1067   // =====
1068   =
1069   // Question: Find Majority Element XXVI
1070   // Input : {10, 20, 30, 40, 50}
1071   // Output: 50
1072   // Time Complexity : O(n)
1073   // Space Complexity: O(1)
1074
1075   // =====
1076   =
1077   public static int findMajorityElementXXVI(int[] arr) {
1078
1079       if (arr == null || arr.length == 0)
1080           throw new IllegalArgumentException("Array is empty");
1081
1082       int largest = arr[0];
1083       int count = 1;
1084
1085       for (int num : arr) {
1086           if (num == largest) {
1087               count++;
1088           } else {
1089               count--;
1090           }
1091
1092           if (count == 0) {
1093               largest = num;
1094               count = 1;
1095           }
1096       }
1097
1098       return largest;
1099   }
1100
1101
1102   // =====
1103   =
1104   // Question: Find Majority Element XXVII
1105   // Input : {10, 20, 30, 40, 50}
1106   // Output: 50
1107   // Time Complexity : O(n)
1108   // Space Complexity: O(1)
1109
1110   // =====
1111   =
1112   public static int findMajorityElementXXVII(int[] arr) {
1113
1114       if (arr == null || arr.length == 0)
1115           throw new IllegalArgumentException("Array is empty");
1116
1117       int largest = arr[0];
1118       int count = 1;
1119
1120       for (int num : arr) {
1121           if (num == largest) {
1122               count++;
1123           } else {
1124               count--;
1125           }
1126
1127           if (count == 0) {
1128               largest = num;
1129               count = 1;
1130           }
1131       }
1132
1133       return largest;
1134   }
1135
1136
1137   // =====
1138   =
1139   // Question: Find Majority Element XXVIII
1140   // Input : {10, 20, 30, 40, 50}
1141   // Output: 50
1142   // Time Complexity : O(n)
1143   // Space Complexity: O(1)
1144
1145   // =====
1146   =
1147   public static int findMajorityElementXXVIII(int[] arr) {
1148
1149       if (arr == null || arr.length == 0)
1150           throw new IllegalArgumentException("Array is empty");
1151
1152       int largest = arr[0];
1153       int count = 1;
1154
1155       for (int num : arr) {
1156           if (num == largest) {
1157               count++;
1158           } else {
1159               count--;
1160           }
1161
1162           if (count == 0) {
1163               largest = num;
1164               count = 1;
1165           }
1166       }
1167
1168       return largest;
1169   }
1170
1171
1172   // =====
1173   =
1174   // Question: Find Majority Element XXIX
1175   // Input : {10, 20, 30, 40, 50}
1176   // Output: 50
1177   // Time Complexity : O(n)
1178   // Space Complexity: O(1)
1179
1180   // =====
1181   =
1182   public static int findMajorityElementXXIX(int[] arr) {
1183
1184       if (arr == null || arr.length == 0)
1185           throw new IllegalArgumentException("Array is empty");
1186
1187       int largest = arr[0];
1188       int count = 1;
1189
1190       for (int num : arr) {
1191           if (num == largest) {
1192               count++;
1193           } else {
1194               count--;
1195           }
1196
1197           if (count == 0) {
1198               largest = num;
1199               count = 1;
1200           }
1201       }
1202
1203       return largest;
1204   }
1205
1206
1207   // =====
1208   =
1209   // Question: Find Majority Element XXX
1210   // Input : {10, 20, 30, 40, 50}
1211   // Output: 50
1212   // Time Complexity : O(n)
1213   // Space Complexity: O(1)
1214
1215   // =====
1216   =
1217   public static int findMajorityElementXXX(int[] arr) {
1218
1219       if (arr == null || arr.length == 0)
1220           throw new IllegalArgumentException("Array is empty");
1221
1222       int largest = arr[0];
1223       int count = 1;
1224
1225       for (int num : arr) {
1226           if (num == largest) {
1227               count++;
1228           } else {
1229               count--;
1230           }
1231
1232           if (count == 0) {
1233               largest = num;
1234               count = 1;
1235           }
1236       }
1237
1238       return largest;
1239   }
1240
1241
1242   // =====
1243   =
1244   // Question: Find Majority Element XXXI
1245   // Input : {10, 20, 30, 40, 50}
1246   // Output: 50
1247   // Time Complexity : O(n)
1248   // Space Complexity: O(1)
1249
1250   // =====
1251   =
1252   public static int findMajorityElementXXXI(int[] arr) {
1253
1254       if (arr == null || arr.length == 0)
1255           throw new IllegalArgumentException("Array is empty");
1256
1257       int largest = arr[0];
1258       int count = 1;
1259
1260       for (int num : arr) {
1261           if (num == largest) {
1262               count++;
1263           } else {
1264               count--;
1265           }
1266
1267           if (count == 0) {
1268               largest = num;
1269               count = 1;
1270           }
1271       }
1272
1273       return largest;
1274   }
1275
1276
1277   // =====
1278   =
1279   // Question: Find Majority Element XXXII
1280   // Input : {10, 20, 30, 40, 50}
1281   // Output: 50
1282   // Time Complexity : O(n)
1283   // Space Complexity: O(1)
1284
1285   // =====
1286   =
1287   public static int findMajorityElementXXXII(int[] arr) {
1288
1289       if (arr == null || arr.length == 0)
1290           throw new IllegalArgumentException("Array is empty");
1291
1292       int largest = arr[0];
1293       int count = 1;
1294
1295       for (int num : arr) {
1296           if (num == largest) {
1297               count++;
1298           } else {
1299               count--;
1300           }
1301
1302           if (count == 0) {
1303               largest = num;
1304               count = 1;
1305           }
1306       }
1307
1308       return largest;
1309   }
1310
1311
1312   // =====
1313   =
1314   // Question: Find Majority Element XXXIII
1315   // Input : {10, 20, 30, 4
```

```

50         for (int num : arr) {
51             if (num < smallest) {
52                 smallest = num;
53             }
54         }
55
56         return smallest;
57     }
58
59
60     // =====
61     ==
62     // Question: Find Second Largest Element
63     // Input : {10, 20, 30, 40, 50}
64     // Output: 40
65     // Time Complexity : O(n)
66     // Space Complexity: O(1)
67
68     // =====
69     ==
70     public static int findSecondLargest(int[] arr) {
71
72         if (arr == null || arr.length < 2)
73             throw new IllegalArgumentException("Array must have at least 2
74 elements");
75
76         int largest      = Integer.MIN_VALUE;
77         int secondLargest = Integer.MIN_VALUE;
78
79         for (int num : arr) {
80             if (num > largest) {
81                 secondLargest = largest;
82                 largest = num;
83             } else if (num > secondLargest && num != largest) {
84                 secondLargest = num;
85             }
86         }
87
88         if (secondLargest == Integer.MIN_VALUE)
89             throw new IllegalArgumentException("No second largest element
90 exists");
91
92         return secondLargest;
93     }
94
95
96     // =====
97     ==
98     // Question: Find Second Smallest Element
99     // Input : {10, 20, 30, 40, 50}
100    // Output: 20
101    // Time Complexity : O(n)
102    // Space Complexity: O(1)
103
104    // =====
105    ==
106    public static int findSecondSmallest(int[] arr) {

```

```

97
98     if (arr == null || arr.length < 2)
99         throw new IllegalArgumentException("Array must have at least 2
elements");
100
101     int smallest      = Integer.MAX_VALUE;
102     int secondSmallest = Integer.MAX_VALUE;
103
104     for (int num : arr) {
105         if (num < smallest) {
106             secondSmallest = smallest;
107             smallest = num;
108         } else if (num < secondSmallest && num != smallest) {
109             secondSmallest = num;
110         }
111     }
112
113     if (secondSmallest == Integer.MAX_VALUE)
114         throw new IllegalArgumentException("No second smallest element
exists");
115
116     return secondSmallest;
117 }
118
119
// =====
==
120 // Question: Reverse Array
121 // Input : {1, 2, 3, 4, 5}
122 // Output: {5, 4, 3, 2, 1}
123 // Time Complexity : O(n)
124 // Space Complexity: O(1)
125
// =====
==
126 public static void reverseArray(int[] arr) {
127
128     if (arr == null || arr.length <= 1) return;
129
130     int left = 0;
131     int right = arr.length - 1;
132
133     while (left < right) {
134         int temp = arr[left];
135         arr[left] = arr[right];
136         arr[right] = temp;
137         left++;
138         right--;
139     }
140 }
141
142
// =====
==
143 // Question: Sum Of Array Elements
144 // Input : {1, 2, 3, 4, 5}
145 // Output: 15

```

```

146    // Time Complexity : O(n)
147    // Space Complexity: O(1)
148
149    // =====
150    ==
151    public static int sumOfElements(int[] arr) {
152
153        if (arr == null) return 0;
154
155        int sum = 0;
156        for (int num : arr) {
157            sum += num;
158        }
159
160        return sum;
161    }
162
163    // =====
164    ==
165    // Question: Average Of Array Elements
166    // Input : {1, 2, 3, 4, 5}
167    // Output: 3.0
168    // Time Complexity : O(n)
169    // Space Complexity: O(1)
170
171    // =====
172    ==
173    public static double averageOfElements(int[] arr) {
174
175        if (arr == null || arr.length == 0) return 0.0;
176
177        return (double) sumOfElements(arr) / arr.length;
178    }
179
180    // =====
181    ==
182    // Question: Count Even And Odd Numbers
183    // Input : {1, 2, 3, 4, 5, 6}
184    // Output: Even = 3, Odd = 3
185    // Time Complexity : O(n)
186
187    // =====
188    ==
189    public static void countEvenOdd(int[] arr) {
190
191        int even = 0;
192        int odd = 0;
193
194        if (arr == null) {
195            System.out.println("Even Count : " + even);
196            System.out.println("Odd Count : " + odd);
197            return;
198        }
199
200        for (int num : arr) {

```



```

193         if (num % 2 == 0) {
194             even++;
195         } else {
196             odd++;
197         }
198     }
199
200     // long even = Arrays.stream(arr)
201     //     .filter(num -> num % 2 == 0)
202     //     .count();
203     //
204     // long odd = Arrays.stream(arr)
205     //     .filter(num -> num % 2 != 0)
206     //     .count();
207
208     // Map<Boolean, Long> counts = Arrays.stream(arr)
209     //     .boxed()
210     //     .collect(Collectors.partitioningBy(
211     //         x -> x % 2 == 0,
212     //         Collectors.counting()
213     //     ));
214     //
215     // long even = counts.get(true);
216     // long odd = counts.get(false);
217
218     System.out.println("    Even Count : " + even);
219     System.out.println("    Odd Count  : " + odd);
220 }
221
222
223 // =====
224 ==
225 // Question: Find Maximum And Minimum
226 // Input : {10, 20, 30, 40, 50}
227 // Output: Max = 50, Min = 10
228 // Time Complexity : O(n)
229 // Space Complexity: O(1)
230
231 // =====
232 ==
233
234 public static int[] findMaxAndMin(int[] arr) {
235
236     if (arr == null || arr.length == 0)
237         throw new IllegalArgumentException("Array is empty");
238
239     int max = arr[0];
240     int min = arr[0];
241
242     for (int num : arr) {
243         if (num > max)
244             max = num;
245         if (num < min)
246             min = num;
247     }
248
249     return new int[]{max, min};
250 }

```

```

246
247
    // =====
    ==
248    // Question: Search Element In Array
249    // Input : {10, 20, 30, 40, 50}, target = 30
250    // Output: true
251    // Time Complexity : O(n)
252    // Space Complexity: O(1)
253
    // =====
    ==
254    public static boolean searchElement(int[] arr, int target) {
255
256        if (arr == null)
257            return false;
258
259        for (int num : arr) {
260            if (num == target)
261                return true;
262        }
263
264        return false;
265    }
266
267
    // =====
    ==
268    // Question: Remove Duplicates From Array
269    // Input : {1, 2, 2, 3, 4, 4, 5}
270    // Output: [1, 2, 3, 4, 5]
271    // Time Complexity : O(n)
272    // Space Complexity: O(n)
273
    // =====
    ==
274    public static int[] removeDuplicates(int[] arr) {
275
276        if (arr == null) return new int[0];
277
278        return Arrays.stream(arr).distinct().toArray();
279    }
280
281
    // =====
    ==
282    // Question: Find Duplicate Elements
283    // Input : {1, 2, 2, 3, 4, 4, 5}
284    // Output: [2, 4]
285    // Time Complexity : O(n)
286    // Space Complexity: O(n)
287
    // =====
    ==
288    public static Set<Integer> findDuplicates(int[] arr) {
289
290        Set<Integer> set = new HashSet<>();

```

```

291         Set<Integer> duplicates = new LinkedHashSet<>();
292
293         if (arr == null)
294             return duplicates;
295
296         for (int num : arr) {
297             if (!set.add(num)) {
298                 duplicates.add(num);
299             }
300         }
301
302         return duplicates;
303     }
304
305
306 // =====
307 ==
308 // Question: Frequency Of Elements
309 // Input : {1, 2, 2, 3, 3, 3}
310 // Output: {1=1, 2=2, 3=3}
311 // Time Complexity : O(n)
312 // Space Complexity: O(n)
313
314 // =====
315 ==
316 public static Map<Integer, Integer> frequencyCount(int[] arr) {
317
318     Map<Integer, Integer> map = new LinkedHashMap<>();
319
320     if (arr == null)
321         return map;
322
323     for (int num : arr) {
324         map.put(num, map.getOrDefault(num, 0) + 1);
325     }
326
327     // First non-repeating element
328     for (Map.Entry<Integer, Integer> entry : map.entrySet()) {
329         if (entry.getValue() == 1) {
330             System.out.println("First Non-Repeating Element: " + entry.
331                 getKey());
332             break;
333         }
334     }
335
336     // Max Count and Element
337     int maxCount = 0;
338     int maxElement = 0;
339
340     for (Map.Entry<Integer, Integer> entry : map.entrySet()) {
341         if (entry.getValue() > maxCount) {
342             maxCount = entry.getValue();
343             maxElement = entry.getKey();
344         }
345     }
346
347     // MajorityElement in an Array n/2

```

```

343 //      if (maxCount > arr.length / 2) {
344 //          System.out.println("Majority Element: " + maxElement);
345 //      } else {
346 //          System.out.println("No Majority Element");
347 //      }
348
349      return map;
350  }
351
352
353  // =====
354  ==
355  // Question: Missing Numbers In Array
356  // Input : {1, 2, 4, 7, 10}
357  // Output: [3, 5, 6, 8, 9]
358  // Time Complexity : O(n)
359  // Space Complexity: O(n)
360
361  // =====
362  ==
363  public static List<Integer> findAllMissingNumbers(int[] arr) {
364
365      List<Integer> result = new ArrayList<>();
366
367      if (arr == null || arr.length == 0) {
368          return result;
369      }
370
371      Set<Integer> set = new HashSet<>();
372
373      int min = Integer.MAX_VALUE;
374      int max = Integer.MIN_VALUE;
375
376      for (int num : arr) {
377          set.add(num);
378          min = Math.min(min, num);
379          max = Math.max(max, num);
380      }
381
382      for (int i = min; i <= max; i++) {
383          if (!set.contains(i)) {
384              result.add(i);
385          }
386      }
387
388      return result;
389  }
390
391  // =====
392  ==
393  // Question: Move Zeros To End
394  // Input : {0, 1, 0, 3, 12}
395  // Output: {1, 3, 12, 0, 0}
396  // Time Complexity : O(n)
397  // Space Complexity: O(1)
398

```

```

393 // =====
394 // For left all condition will be -- nonZeroPos = nums.length - 1 for loop
    starts nums.length-1, i>= 0, i--
395     public static void moveZerosToEnd(int[] nums) {
396
397         if (nums == null || nums.length <= 1)
398             return;
399
400         int nonZeroPos = 0;
401
402         for (int i = 0; i < nums.length; i++) {
403             if (nums[i] != 0) {
404                 int temp        = nums[i];
405                 nums[i]         = nums[nonZeroPos];
406                 nums[nonZeroPos] = temp;
407                 nonZeroPos++;
408             }
409         }
410     }
411
412 // =====
413 // Question: Merge Two Arrays
414 // Input : {1, 2, 3} and {4, 5, 6}
415 // Output: {1, 2, 3, 4, 5, 6}
416 // Time Complexity : O(n + m)
417 // Space Complexity: O(n + m)
418
419 // =====
420 //
421 // public static int[] mergeArrays(int[] arr1, int[] arr2) {
422 //
423 //     int[] result = new int[arr1.length + arr2.length];
424 //
425 //     for (int i = 0; i < arr1.length; i++) {
426 //         result[i] = arr1[i];
427 //     }
428 //
429 //     for (int i = 0; i < arr2.length; i++) {
430 //         result[arr1.length + i] = arr2[i];
431 //     }
432 //
433 //     return result;
434 //
435 //     return IntStream.concat(
436 //         Arrays.stream(arr1),
437 //         Arrays.stream(arr2)
438 //     ).toArray();
439 // }
440 // =====
441 // Question: Intersection Of Arrays

```

```

441 // Input : {1, 2, 3, 4, 5} and {4, 5, 6, 7}
442 // Output: {4, 5}
443 // Time Complexity :  $O(n + m)$ 
444 // Space Complexity:  $O(n)$ 
445
446 // =====
447 ==
448 public static int[] intersection(int[] arr1, int[] arr2) {
449
450     if (arr1 == null || arr2 == null) return new int[0];
451
452     Set<Integer> set = Arrays.stream(arr1).boxed().collect(Collectors.
toSet());
453
454     return Arrays.stream(arr2)
455         .distinct()
456         .filter(set::contains)
457         .toArray();
458 }
459
460 // =====
461 ==
462 // Question: Union Of Arrays
463 // Input : {1, 2, 3, 4, 5} and {4, 5, 6, 7}
464 // Output: {1, 2, 3, 4, 5, 6, 7}
465 // Time Complexity :  $O(n + m)$ 
466 // Space Complexity:  $O(n + m)$ 
467
468 // =====
469 ==
470 public static int[] union(int[] arr1, int[] arr2) {
471
472     if (arr1 == null) arr1 = new int[0];
473     if (arr2 == null) arr2 = new int[0];
474
475     return IntStream.concat(
476         Arrays.stream(arr1),
477         Arrays.stream(arr2))
478         .distinct()
479         .toArray();
480 }
481
482 // =====
483 ==
484 // Question: Two Sum Problem
485 // Input : {2, 7, 11, 15}, target = 9
486 // Output: [0, 1] (indices where arr[0]+arr[1] = 9)
487 // Time Complexity :  $O(n^2)$ 
488 // Space Complexity:  $O(1)$ 
489
490 // =====
491 ==
492 public static List<int[]> allPairs(int[] nums, int target) {
493
494     List<int[]> result = new ArrayList<>();

```



```

487
488     for (int i = 0; i < nums.length; i++) {
489         for (int j = i + 1; j < nums.length; j++) {
490             if (nums[i] + nums[j] == target) {
491                 result.add(new int[]{i, j});
492             }
493         }
494     }
495
496     return result;
497 }
498
499
// =====
==
500 // Question: Pair With Given Sum (Using HashSet)
501 // Input : {1, 2, 3, 4, 5}, target = 7
502 // Output: [2,5], [3,4]
503 // Time Complexity : O(n)
504 // Space Complexity: O(n)
505
// =====
==
506 public static List<int[]> findPairsWithSum(int[] arr, int target) {
507
508     List<int[]> result = new ArrayList<>();
509     Set<Integer> set = new HashSet<>();
510
511     for (int num : arr) {
512
513         int complement = target - num;
514
515         if (set.contains(complement)) {
516             result.add(new int[]{complement, num});
517         }
518
519         set.add(num);
520     }
521
522     return result;
523 }
524
525
// =====
==
526 // Question: Top K Frequent Elements
527 // Input : [1,1,1,2,2,3], k = 2
528 // Output: [1,2]
529 //
530 // Explanation:
531 // 1. Count the frequency using HashMap.
532 // 2. Sort the map entries by frequency in descending order.
533 // 3. Return the first k elements.
534 //
535 // Time Complexity : O(n log n)
536 // Space Complexity: O(n)
537

```

```

537 // =====
538 ==
539 public static List<Integer> topKFrequent(int[] nums, int k) {
540     Map<Integer, Integer> map = new HashMap<>();
541
542     // Count frequency
543     for (int num : nums) {
544         map.put(num, map.getOrDefault(num, 0) + 1);
545     }
546
547     List<Map.Entry<Integer, Integer>> list = new ArrayList<>(map.entrySet
548     ());
549
550     // Sort by frequency (highest first)
551     list.sort((a, b) -> b.getValue() - a.getValue());
552
553     List<Integer> result = new ArrayList<>();
554
555     for (int i = 0; i < k; i++) {
556         result.add(list.get(i).getKey());
557     }
558
559     return result;
560 }
561
562 // =====
563 ==
564 // Question: Maximum Subarray Sum (Kadane's Algorithm)
565 // Input : {-2, 1, -3, 4, -1, 2, 1, -5, 4}
566 // Output: 6
567 // Subarray = {4, -1, 2, 1}
568 // Time Complexity : O(n)
569 // Space Complexity: O(1)
570
571 // =====
572 ==
573 public static void maxSubArray(int[] arr) {
574
575     int currentSum = arr[0];
576     int maxSum = arr[0];
577
578     int start = 0;
579     int end = 0;
580     int tempStart = 0;
581     for (int i = 1; i < arr.length; i++) {
582
583         if (arr[i] > currentSum + arr[i]) {
584             // Forget Prev array start with new array
585             currentSum = arr[i];
586             tempStart = i;
587         } else {
588             // Prev array is imp continue
589             currentSum += arr[i];
590         }
591     }

```



```

587
588         if (currentSum > maxSum) {
589             maxSum = currentSum;
590             start = tempStart;
591             end = i;
592         }
593     }
594     System.out.println("Max Sum = " + maxSum);
595
596     for (int i = start; i <= end; i++) {
597         System.out.print(arr[i] + " ");
598     }
599 }
600
601
602 // =====
603 ==
604 // Question: Subset Sum (Combination Sum)
605 // Input : nums = {2, 7, 3, 6, 4, 5}, target = 9
606 // Output: All combinations that sum to 9
607 //          e.g. [2, 7], [3, 6], [4, 5], [2, 3, 4] ...
608
609 // =====
610 ==
611 public static class SubsetSum {
612
613     public static void findCombinations(int[] nums, int target) {
614         System.out.println("    Combinations that sum to " + target + ":");
615     };
616
617     backtrack(nums, target, 0, new ArrayList<>());
618 }
619
620 private static void backtrack(int[] nums,
621                               int target,
622                               int start,
623                               List<Integer> current) {
624
625     if (target == 0) {
626         System.out.println("    " + current);
627         return;
628     }
629
630     if (target < 0)
631         return;
632
633     for (int i = start; i < nums.length; i++) {
634         current.add(nums[i]);
635         backtrack(nums, target - nums[i], i + 1, current);
636         current.remove(current.size() - 1);
637     }
638 }
639
640 }
641
642 // =====
643 ==
644 // Question: Container With Most Water

```

```

637 // Input : [1,8,6,2,5,4,8,3,7]
638 // Output: 49
639 //
640 // Explanation:
641 // Find two lines that together with the x-axis form a container
642 // that holds the maximum amount of water.
643 //
644 // Formula:
645 // Area = min(height[left], height[right]) * (right - left)
646 //
647 // Time Complexity : O(n)
648 // Space Complexity: O(1)
649
// =====
==
650 public static int maxArea(int[] height) {
651
652     int left = 0;
653     int right = height.length - 1;
654
655     int maxArea = 0;
656
657     while (left < right) {
658
659         int area = Math.min(height[left], height[right]) * (right - left
660 );
661
662         maxArea = Math.max(maxArea, area);
663
664         if (height[left] < height[right]) {
665             left++;
666         } else {
667             right--;
668         }
669
670         return maxArea;
671     }
672
673
// =====
==
674 // Question: Remove Duplicates from Sorted Array
675 // Input : [1,1,2,2,3,4,4]
676 // Output: [1,2,3,4]
677 //
678 // Explanation:
679 // Remove duplicates in-place from a sorted array and return
680 // the number of unique elements.
681 //
682 // Time Complexity : O(n)
683 // Space Complexity: O(1)
684
// =====
==
685 public static int removeDuplicateTwoPointers(int[] nums) {
686

```

```

687         if (nums.length == 0) {
688             return 0;
689         }
690
691         int i = 0;
692
693         for (int j = 1; j < nums.length; j++) {
694
695             if (nums[i] != nums[j]) {
696                 i++;
697                 nums[i] = nums[j];
698             }
699         }
700
701         return i + 1;
702     }
703
704
705     // =====
706     ==
707     // Question: Search Insert Position
708     // Input : nums = [1,3,5,6], target = 5
709     // Output: 2
710     //
711     // Explanation:
712     // Return the index if the target is found.
713     // Otherwise, return the index where it should be inserted.
714     //
715     // Time Complexity : O(log n)
716     // Space Complexity: O(1)
717
718     // =====
719     ==
720     public static int searchInsert(int[] nums, int target) {
721
722         int left = 0;
723         int right = nums.length - 1;
724
725         while (left <= right) {
726
727             int mid = left + (right - left) / 2;
728
729             if (nums[mid] == target) {
730                 return mid;
731             } else if (nums[mid] < target) {
732                 left = mid + 1;
733             } else {
734                 right = mid - 1;
735             }
736         }
737
738         return left;
739     }
740
741     // =====
742     ==

```

```

738 // Question: First Occurrence of an Element
739 // Input : nums = [1,2,2,2,3,4], target = 2
740 // Output: 1
741 //
742 // Explanation:
743 // Find the first occurrence of the target in a sorted array.
744 //
745 // Time Complexity : O(log n)
746 // Space Complexity: O(1)
747
// =====
==
748 public static int firstOccurrence(int[] nums, int target) {
749
750     int left = 0;
751     int right = nums.length - 1;
752     int result = -1;
753
754     while (left <= right) {
755
756         int mid = left + (right - left) / 2;
757
758         if (nums[mid] == target) {
759             result = mid;
760             right = mid - 1; // Search on left side
761         } else if (nums[mid] < target) {
762             left = mid + 1;
763         } else {
764             right = mid - 1;
765         }
766     }
767
768     return result;
769 }
770
771
// =====
==
772 // Question: Last Occurrence of an Element
773 // Input : nums = [1,2,2,2,3,4], target = 2
774 // Output: 3
775 //
776 // Explanation:
777 // Find the last occurrence of the target in a sorted array.
778 //
779 // Time Complexity : O(log n)
780 // Space Complexity: O(1)
781
// =====
==
782 public static int lastOccurrence(int[] nums, int target) {
783
784     int left = 0;
785     int right = nums.length - 1;
786     int result = -1;
787
788     while (left <= right) {

```

```

789
790         int mid = left + (right - left) / 2;
791
792         if (nums[mid] == target) {
793             result = mid;
794             left = mid + 1;          // Search on right side
795         } else if (nums[mid] < target) {
796             left = mid + 1;
797         } else {
798             right = mid - 1;
799         }
800     }
801
802     return result;
803 }
804
805
// =====
==
806 // Question: Find Peak Element
807 // Input : [1,2,3,1]
808 // Output: 2
809 //
810 // Explanation:
811 // nums[2] = 3 is greater than its neighbors (2 and 1),
812 // so index 2 is the peak element.
813 //
814 // Time Complexity : O(log n)
815 // Space Complexity: O(1)
816
// =====
==
817 public static int findPeakElement(int[] nums) {
818
819     int left = 0;
820     int right = nums.length - 1;
821
822     while (left < right) {
823
824         int mid = left + (right - left) / 2;
825
826         if (nums[mid] < nums[mid + 1]) {
827             left = mid + 1;
828         } else {
829             right = mid;
830         }
831     }
832
833     return left;
834 }
835
836
// =====
==
837 // Question: Square Root (Integer)
838 // Input : 8
839 // Output: 2

```

```

840    //
841    // Explanation:
842    // Return the integer part (floor) of the square root.
843    //
844    // sqrt(8) = 2.828...
845    // Answer = 2
846    //
847    // Time Complexity : O(log n)
848    // Space Complexity: O(1)
849
    // =====
    ==
850    public static int mySqrt(int x) {
851
852        if (x == 0 || x == 1) {
853            return x;
854        }
855
856        int left = 1;
857        int right = x;
858        int ans = 0;
859
860        while (left <= right) {
861
862            int mid = left + (right - left) / 2;
863
864            long square = (long) mid * mid;
865
866            if (square == x) {
867                return mid;
868            } else if (square < x) {
869                ans = mid;
870                left = mid + 1;
871            } else {
872                right = mid - 1;
873            }
874        }
875
876        return ans;
877    }
878
879
    // =====
    ==
880    // Question: Next Greater Element (Brute Force)
881    // Input : [4, 5, 2, 10, 8]
882    // Output: [5, 10, 10, -1, -1]
883    //
884    // Explanation:
885    // For every element, check all elements on its right.
886    // The first greater element is the answer.
887    // If none is found, return -1.
888    //
889    // Time Complexity : O(n2)
890    // Space Complexity: O(1)
891
    // =====

```



```

891 ==
892     public static int[] nextGreaterElement(int[] arr) {
893
894         int[] result = new int[arr.length];
895
896         for (int i = 0; i < arr.length; i++) {
897
898             result[i] = -1;
899
900             for (int j = i + 1; j < arr.length; j++) {
901
902                 if (arr[j] > arr[i]) {
903                     result[i] = arr[j];
904                     break;
905                 }
906             }
907         }
908
909         return result;
910     }
911
912
913 // =====
914 ==
915 // Question: Leaders In Array (Brute Force)
916 // Input : {16, 17, 4, 3, 5, 2}
917 // Output: [17, 5, 2]
918 //
919 // Explanation:
920 // A leader is an element that is greater than all the elements
921 // to its right.
922 //
923 // Time Complexity :  $O(n^2)$ 
924 // Space Complexity:  $O(1)$ 
925
926 // =====
927 ==
928     public static List<Integer> findLeaders(int[] arr) {
929
930         List<Integer> leaders = new ArrayList<>();
931
932         for (int i = 0; i < arr.length; i++) {
933
934             boolean isLeader = true;
935
936             for (int j = i + 1; j < arr.length; j++) {
937
938                 if (arr[j] > arr[i]) {
939                     isLeader = false;
940                     break;
941                 }
942             }
943
944             if (isLeader) {
945                 leaders.add(arr[i]);
946             }
947         }
948     }

```

```

944
945     return leaders;
946 }
947
948
// =====
===
949 // Question: Sort 0s 1s 2s (Dutch National Flag Algorithm)
950 // Input : {2, 0, 2, 1, 1, 0}
951 // Output: {0, 0, 1, 1, 2, 2}
952 // Time Complexity : O(n)
953 // Space Complexity: O(1)
954
// =====
===
955 public static void sortZeroOneTwo(int[] arr) {
956
957     if (arr == null || arr.length <= 1) {
958         return;
959     }
960
961     int zero = 0;
962     int one = 0;
963     int two = 0;
964
965     // Count 0s, 1s, and 2s
966     for (int num : arr) {
967         if (num == 0) {
968             zero++;
969         } else if (num == 1) {
970             one++;
971         } else if (num == 2) {
972             two++;
973         } else {
974             throw new IllegalArgumentException(
975                 "Array should contain only 0, 1, and 2");
976         }
977     }
978
979     // Fill the array back
980     int index = 0;
981
982     for (int i = 0; i < zero; i++) {
983         arr[index++] = 0;
984     }
985
986     for (int i = 0; i < one; i++) {
987         arr[index++] = 1;
988     }
989
990     for (int i = 0; i < two; i++) {
991         arr[index++] = 2;
992     }
993
994     System.out.println(Arrays.toString(arr));
995 }
996

```



```

997
    // =====
    ===
998    // Question: Product Of Array Except Self
999    // Input : {1, 2, 3, 4}
1000    // Output: {24, 12, 8, 6}
1001    // (result[i] = product of all elements except arr[i])
1002    // Time Complexity : O(n^2)
1003    // Space Complexity: O(n)
1004
    // =====
    ===
1005    public static int[] productExceptSelf(int[] nums) {
1006
1007        int totalProduct = 1;
1008
1009        for (int num : nums) {
1010            totalProduct *= num;
1011        }
1012
1013        int[] result = new int[nums.length];
1014
1015        for (int i = 0; i < nums.length; i++) {
1016            result[i] = totalProduct / nums[i];
1017        }
1018
1019        return result;
1020    }
1021
1022
    // =====
    ===
1023    // Question: Best Time To Buy And Sell Stock
1024    // Input : {7, 1, 5, 3, 6, 4}
1025    // Output: 5
1026    // Buy = 1 (buy at lowest price)
1027    // Sell = 6 (sell at highest price after buy)
1028    // Profit = Sell - Buy = 6 - 1 = 5
1029    // Time Complexity : O(n)
1030    // Space Complexity: O(1)
1031
    // =====
    ===
1032    public static int maxProfit(int[] prices) {
1033
1034        if (prices == null || prices.length <= 1) return 0;
1035
1036        int minPrice = prices[0];
1037        int buyDay = 0;
1038        int sellDay = 0;
1039        int maxProfit = 0;
1040
1041        for (int i = 1; i < prices.length; i++) {
1042
1043            if (prices[i] < minPrice) {
1044                minPrice = prices[i];
1045                buyDay = i;

```

```

1046         }
1047
1048         if (prices[i] - minPrice > maxProfit) {
1049             maxProfit = prices[i] - minPrice;
1050             sellDay = i;
1051         }
1052     }
1053
1054     System.out.println("    Buy Day   : Day " + buyDay + " -> Price
= " + prices[buyDay]);
1055     System.out.println("    Sell Day    : Day " + sellDay + " -> Price
= " + prices[sellDay]);
1056     System.out.println("    Max Profit : " + prices[sellDay] + " - " +
prices[buyDay] + " = " + maxProfit);
1057
1058     return maxProfit;
1059 }
1060
1061
1062 // =====
1063 ==
1064 // Question: First Non-Repeating Element (Already done above)
1065 // Input : {9, 4, 9, 6, 7, 4}
1066 // Output: 6
1067 // Time Complexity : O(n)
1068 // Space Complexity: O(n)
1069
1070 // =====
1071 ==
1072 public static Integer firstNonRepeating(int[] arr) {
1073
1074     if (arr == null || arr.length == 0) return null;
1075
1076     Map<Integer, Integer> map = new LinkedHashMap<>();
1077
1078     for (int num : arr) {
1079         map.put(num, map.getOrDefault(num, 0) + 1);
1080     }
1081
1082     for (Map.Entry<Integer, Integer> entry : map.entrySet()) {
1083         if (entry.getValue() == 1) {
1084             return entry.getKey();
1085         }
1086     }
1087
1088     return null;
1089 }
1090
1091 // =====
1092 ==
1093 // Question: Contains Duplicate
1094 // Input : {1, 2, 3, 1}
1095 // Output: true
1096 // Time Complexity : O(n)
1097 // Space Complexity: O(n)
1098

```

```

1093
    // =====
    ===
1094    public static boolean containsDuplicate(int[] arr) {
1095
1096        if (arr == null || arr.length <= 1) return false;
1097
1098        return Arrays.stream(arr).distinct().count() != arr.length;
1099    }
1100
1101
    // =====
    ===
1102    // Question: Sort Array Ascending (Using Stream)
1103    // Input : {5, 3, 8, 1, 2}
1104    // Output: {1, 2, 3, 5, 8}
1105
    // =====
    ===
1106    public static int[] sortAscendingStream(int[] arr) {
1107
1108        if (arr == null) return null;
1109
1110        return Arrays.stream(arr).sorted().toArray();
1111    }
1112
1113
    // =====
    ===
1114    // Question: Sort Array Descending (Using Stream)
1115    // Input : {5, 3, 8, 1, 2}
1116    // Output: {8, 5, 3, 2, 1}
1117
    // =====
    ===
1118    public static int[] sortDescendingStream(int[] arr) {
1119
1120        if (arr == null) return null;
1121
1122        // Integer[] arr = {5, 3, 8, 1, 2};
1123        //
1124        // Arrays.sort(arr, Collections.reverseOrder());
1125        //
1126        // System.out.println(Arrays.toString(arr));
1127
1128        return Arrays.stream(arr)
1129            .boxed()
1130            .sorted(Collections.reverseOrder())
1131            .mapToInt(Integer::intValue)
1132            .toArray();
1133    }
1134
1135
    // =====
    ===
1136    // Question: Separate Odd and Even Numbers (Using Stream)
1137    // Input : {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}

```

```

1138    // Output: Even = [2, 4, 6, 8, 10]  Odd = [1, 3, 5, 7, 9]
1139
1140    // =====
1141    ===
1142    public static void findOddEvenNumbers(int[] nums) {
1143
1144        if (nums == null) return;
1145
1146        List<Integer> evenNumbers = Arrays.stream(nums)
1147            .boxed()
1148            .filter(n -> n % 2 == 0)
1149            .collect(Collectors.toList());
1150
1151        List<Integer> oddNumbers = Arrays.stream(nums)
1152            .boxed()
1153            .filter(n -> n % 2 != 0)
1154            .collect(Collectors.toList());
1155
1156        System.out.println("    Even Numbers : " + evenNumbers);
1157        System.out.println("    Odd Numbers  : " + oddNumbers);
1158    }
1159
1160    // =====
1161    ===
1162    // Question: Sort Map By Values
1163    // Input : {A=3, B=1, C=2, D=1}
1164    // Output: {B=1, D=1, C=2, A=3}
1165
1166    // =====
1167    ===
1168    public static void sortMapByValues(Map<String, Integer> map) {
1169
1170        if (map == null) return;
1171
1172        Map<String, Integer> sortedMap = map.entrySet()
1173            .stream()
1174            .sorted(Map.Entry.comparingByValue())
1175            .collect(Collectors.toMap(
1176                Map.Entry::getKey,
1177                Map.Entry::getValue,
1178                (oldValue, newValue) -> oldValue,
1179                LinkedHashMap::new
1180            ));
1181
1182        System.out.println("    Sorted Map : " + sortedMap);
1183    }
1184
1185    // =====
1186    ===
1187    // Question: Sort Map By Value Then Key
1188    // Input : {C=2, A=1, D=2, B=1}
1189    // Output: {A=1, B=1, C=2, D=2}
1190
1191    // =====
1192    ===

```

```

1185     public static void sortMapByValueThenKey(Map<String, Integer> map) {
1186
1187         if (map == null) return;
1188
1189         Map<String, Integer> sortedMap = map.entrySet()
1190             .stream()
1191             .sorted(
1192                 Map.Entry.<String, Integer>comparingByValue()
1193                     .thenComparing(Map.Entry.comparingByKey())
1194             )
1195             .collect(Collectors.toMap(
1196                 Map.Entry::getKey,
1197                 Map.Entry::getValue,
1198                 (oldValue, newValue) -> oldValue,
1199                 LinkedHashMap::new
1200             ));
1201
1202         System.out.println("    Sorted Map : " + sortedMap);
1203     }
1204
1205
1206
1207     // =====
1208     // Question: Rotate Array By K Positions (Right Rotation)
1209     // Input : {1, 2, 3, 4, 5, 6, 7}, k = 3
1210     // Output: {5, 6, 7, 1, 2, 3, 4}
1211     // Time Complexity : O(n)
1212     // Space Complexity: O(1)
1213
1214     // =====
1215     //
1216     public static void rotateArrayByK(int[] nums, int k) {
1217
1218         if (nums == null || nums.length == 0 || k < 0)
1219             return;
1220
1221         k = k % nums.length;
1222         reverseSubArray(nums, 0, nums.length - 1);
1223         reverseSubArray(nums, 0, k - 1);
1224         reverseSubArray(nums, k, nums.length - 1);
1225     }
1226
1227     // =====
1228     // Question: Rotate Array By K Positions (Left Rotation)
1229     // Input : {1, 2, 3, 4, 5, 6, 7}, k = 3
1230     // Output: {4, 5, 6, 7, 1, 2, 3}
1231     // Time Complexity : O(n)
1232     // Space Complexity: O(1)
1233
1234     // =====
1235     //
1236     public static void rotateArrayLeftByK(int[] nums, int k) {
1237
1238         if (nums == null || nums.length == 0 || k < 0)

```

```

1234         return;
1235
1236         k = k % nums.length;
1237
1238         reverseSubArray(nums, 0, k - 1);
1239         reverseSubArray(nums, k, nums.length - 1);
1240         reverseSubArray(nums, 0, nums.length - 1);
1241     }
1242
1243     private static void reverseSubArray(int[] nums, int start, int end) {
1244         while (start < end) {
1245             int temp = nums[start];
1246             nums[start] = nums[end];
1247             nums[end] = temp;
1248             start++;
1249             end--;
1250         }
1251     }
1252
1253
1254     // =====
1255     ===
1256     // Question: Find Missing Single Number (1 to N sequence)
1257     // Input : {1, 2, 4, 5, 6}, n = 6
1258     // Output: 3
1259     // Time Complexity : O(n)
1260     // Space Complexity: O(1)
1261
1262     // =====
1263     ===
1264     public static int findSingleMissingNumber(int[] arr, int n) {
1265
1266         int expectedSum = n * (n + 1) / 2;
1267         int actualSum = 0;
1268
1269         for (int num : arr) {
1270             actualSum += num;
1271         }
1272
1273         return expectedSum - actualSum;
1274     }
1275
1276
1277     // =====
1278     ===
1279     // Question: Find Maximum Consecutive Occurrence of 1
1280     // Input : "110111101"
1281     // Output: 4
1282     // Time Complexity : O(n)
1283     // Space Complexity: O(1)
1284
1285     // =====
1286     ===
1287     public static int findMaxConsecutiveOnes(String str) {
1288
1289         int maxCount = 0;
1290         int currentCount = 0;

```



```

1283
1284     for (char ch : str.toCharArray()) {
1285
1286         if (ch == '1') {
1287             currentCount++;
1288             maxCount = Math.max(maxCount, currentCount);
1289         } else {
1290             currentCount = 0;
1291         }
1292     }
1293
1294     return maxCount;
1295 }
1296
1297
1298 // =====
1299 //
1300 // Question: Flatten Nested List
1301 // Input : [[1, 2], [3, 4, 5]]
1302 // Output: [1, 2, 3, 4, 5]
1303
1304 // =====
1305 //
1306 public static List<Integer> flattenList(List<List<Integer>> nestedList
1307 ) {
1308
1309     if (nestedList == null) return Collections.emptyList();
1310
1311     return nestedList.stream()
1312         .flatMap(List::stream)
1313         .collect(Collectors.toList());
1314 }
1315
1316 // =====
1317 //
1318 // Question: Split Array into Buckets of Fixed Size
1319 // Input : [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], Bucket Size = 3
1320 // Output: [[1, 2, 3], [4, 5, 6], [7, 8, 9], [10]]
1321
1322 // =====
1323 //
1324 List<int[]> buckets = splitArray(arr, bucketSize);
1325 //
1326 for (int[] bucket : buckets) {
1327     System.out.println(Arrays.toString(bucket));
1328 }
1329
1330 public static List<int[]> splitArray(int[] arr, int bucketSize) {
1331     List<int[]> result = new ArrayList<>();
1332
1333     for (int i = 0; i < arr.length; i += bucketSize) {
1334         result.add(Arrays.copyOfRange(
1335             arr,
1336             i,
1337             Math.min(i + bucketSize, arr.length)
1338         ));
1339     }

```

```

1331
1332     return result;
1333 }
1334
1335
1336 // =====
1337 ===
1338 // Question: Check Whether a Palindrome Can Be Formed from a String
1339 // Input : "aabb"
1340 // Output: true
1341 //
1342 // Input : "aabbc"
1343 // Output: true
1344 //
1345 // Input : "abc"
1346 // Output: false
1347
1348 // =====
1349 ===
1350 public static boolean canFormPalindrome(String str) {
1351
1352     Map<Character, Integer> map = new HashMap<>();
1353
1354     for (char ch : str.toCharArray()) {
1355         map.put(ch, map.getOrDefault(ch, 0) + 1);
1356     }
1357
1358     int oddCount = 0;
1359
1360     for (int count : map.values()) {
1361         if (count % 2 != 0) {
1362             oddCount++;
1363         }
1364     }
1365
1366     return oddCount <= 1;
1367 }
1368
1369 // =====
1370 ===
1371 // Question: Minimum Size Subarray Sum
1372 // Input : [2,3,1,2,4,3], Target = 7
1373 // Output: 2
1374 // Explanation: The subarray [4,3] has the minimum length (2)
1375
1376 // =====
1377 ===
1378 public static int minSubArrayLen(int target, int[] nums) {
1379
1380     int left = 0;
1381     int sum = 0;
1382     int minLength = Integer.MAX_VALUE;
1383
1384     for (int right = 0; right < nums.length; right++) {
1385
1386         sum += nums[right];

```



```

1380
1381         while (sum >= target) {
1382
1383             minLength = Math.min(minLength, right - left + 1);
1384
1385             sum -= nums[left];
1386             left++;
1387         }
1388     }
1389
1390     return minLength == Integer.MAX_VALUE ? 0 : minLength;
1391 }
1392
1393
1394 // =====
1395 ===
1396 // Question: Find Kth Largest Element
1397 // Input : [3,2,1,5,6,4], k = 2
1398 // Output: 5
1399
1400 // =====
1401 ===
1402 public static int findKthLargest(int[] nums, int k) {
1403
1404     Arrays.sort(nums);
1405
1406     return nums[nums.length - k];
1407 }
1408
1409
1410 // =====
1411 ===
1412 // Question: Factorial
1413 // Input : 5
1414 // Output: 120
1415
1416 // =====
1417 ===
1418 public static long factorial(int n) {
1419
1420     if (n < 0)
1421         throw new IllegalArgumentException("Negative numbers not allowed
1422 ");
1423
1424     long factorial = 1;
1425
1426     for (int i = 1; i <= n; i++) {
1427         factorial *= i;
1428     }
1429
1430     return factorial;
1431 }
1432
1433
1434 // =====
1435 ===
1436 // Question: Prime Number

```

```

1426    // Input : 29
1427    // Output: true
1428
1429    // =====
1430    ===
1431    public static boolean isPrime(int n) {
1432        if (n <= 1)
1433            return false;
1434        for (int i = 2; i <= Math.sqrt(n); i++) {
1435            if (n % i == 0)
1436                return false;
1437        }
1438        return true;
1439    }
1440
1441
1442    // =====
1443    ===
1444    // Question: Print Fibonacci Series
1445    // Input : 10
1446    // Output: 0 1 1 2 3 5 8 13 21 34
1447
1448    // =====
1449    ===
1450    public static void printFibonacciSeries(int n) {
1451        int first = 0;
1452        int second = 1;
1453        for (int i = 0; i < n; i++) {
1454            System.out.print(first + " ");
1455            int next = first + second;
1456            first = second;
1457            second = next;
1458        }
1459    }
1460
1461
1462    // =====
1463    ===
1464    // Question: Power (Using Recursion)
1465    // Input : x = 2, n = 5
1466    // Output: 32
1467    //
1468    // Explanation:
1469    // Multiply x recursively until the exponent becomes 0.
1470    //
1471    // Time Complexity : O(n)
1472    // Space Complexity: O(n) (Recursive Call Stack)
1473
1474    // =====
1475    ===

```

```

1473     public static long power(int x, int n) {
1474
1475         if (n == 0) {
1476             return 1;
1477         }
1478
1479         return x * power(x, n - 1);
1480     }
1481
1482     public static void printEvenFibonacciNumbers(int num) {
1483         int first = 0;
1484         int second = 1;
1485         int evenCount = 0;
1486
1487         while (evenCount < num) {
1488             if (first % 2 == 0) {
1489                 System.out.print(first + " ");
1490                 evenCount++;
1491             }
1492
1493             int next = first + second;
1494             first = second;
1495             second = next;
1496         }
1497     }
1498
1499
1500
1501     // =====
1502     // Question: Print Even and Odd Numbers Using Two Threads
1503     // =====
1504
1505     public static void printEvenOddUsingThreads(int max) {
1506
1507         Object lock = new Object();
1508         int[] counter = {1};
1509
1510         Thread oddThread = new Thread(() -> {
1511             synchronized (lock) {
1512                 while (counter[0] <= max) {
1513                     if (counter[0] % 2 == 0) {
1514                         try { lock.wait(); } catch (InterruptedException e
1515 ) { Thread.currentThread().interrupt(); }
1516                     } else {
1517                         System.out.println("    Odd Thread : " + counter[0
1518 ]);
1519                         counter[0]++;
1520                         lock.notify();
1521                     }
1522                 }
1523             }
1524         });
1525
1526         Thread evenThread = new Thread(() -> {
1527             synchronized (lock) {

```

```

1524         while (counter[0] <= max) {
1525             if (counter[0] % 2 != 0) {
1526                 try { lock.wait(); } catch (InterruptedException e
1527             ) { Thread.currentThread().interrupt(); }
1528             } else {
1529                 System.out.println("    Even Thread : " + counter[0
1530             ]);
1531                 counter[0]++;
1532                 lock.notify();
1533             }
1534         }
1535     });
1536     oddThread.start();
1537     evenThread.start();
1538 }
1539
1540
1541 // =====
1542 // Employee Class – used for Stream-based questions below
1543 // =====
1544
1545 public static class Employee {
1546     private String name;
1547     private String department;
1548     private double salary;
1549
1550     public Employee(String name, String department, double salary) {
1551         this.name = name;
1552         this.department = department;
1553         this.salary = salary;
1554     }
1555
1556     public String getName() { return name; }
1557     public String getDepartment() { return department; }
1558     public double getSalary() { return salary; }
1559
1560     @Override
1561     public String toString() {
1562         return name + " (" + salary + ")";
1563     }
1564 }
1565
1566 //          | Requirement          | Collector
1567 //          | ----- | -----
1568 //          | Count employees | `Collectors.counting
1569 //          | Average salary | `Collectors.averagingDouble(Employee::
1570 //          | Total salary | `Collectors.summingDouble(Employee::

```

```

1570 //          | Highest salary | `Collectors.maxBy(Comparator.comparing(
      Employee::getSalary))` |
1571 //          | Lowest salary   | `Collectors.minBy(Comparator.comparing(
      Employee::getSalary))` |
1572
1573
1574
      // =====
      ===
1575      // Question: Group Employees By Department (Stream)
1576
      // =====
      ===
1577      public static Map<String, List<Employee>> groupEmployeesByDept(List<
      Employee> employees) {
1578
1579          if (employees == null) return Collections.emptyMap();
1580
1581          return employees.stream()
1582              .collect(Collectors.groupingBy(Employee::getDepartment));
1583      }
1584
1585
      // =====
      ===
1586      // Question: Highest Salary Employee By Department (Stream)
1587
      // =====
      ===
1588      public static Map<String, Optional<Employee>> highestSalaryByDept(List<
      Employee> employees) {
1589
1590          if (employees == null) return Collections.emptyMap();
1591
1592          return employees.stream()
1593              .collect(Collectors.groupingBy(
1594                  Employee::getDepartment,
1595                  Collectors.maxBy(Comparator.comparing(Employee::
      getSalary))
1596              ));
1597      }
1598
1599
      // =====
      ===
1600      // Question: Employee With Highest Salary
1601
      // =====
      ===
1602      public static Optional<Employee> highestSalary(List<Employee> employees
      ) {
1603
1604          return employees.stream()
1605              .max(Comparator.comparing(Employee::getSalary));
1606      }
1607
1608

```

```

1608
    // =====
    ===
1609    // Question: Second Highest Salary Employee
1610
    // =====
    ===
1611    public static Optional<Employee> secondHighestSalary(List<Employee>
employees) {
1612
1613        return employees.stream()
1614            .sorted(Comparator.comparing(Employee::getSalary).reversed
1615            ())
1616            .skip(1)
1617            .findFirst();
1618    }
1619
    // =====
    ===
1620    // Question: Top 3 Highest Paid Employees
1621
    // =====
    ===
1622    public static List<Employee> top3Employees(List<Employee> employees) {
1623
1624        return employees.stream()
1625            .sorted(Comparator.comparing(Employee::getSalary).reversed
1626            ())
1627            .limit(3)
1628            .collect(Collectors.toList());
1629    }
1630
    // =====
    ===
1631    // Question: Count Employees In Each Department
1632
    // =====
    ===
1633    public static Map<String, Long> countByDepartment(List<Employee>
employees) {
1634
1635        return employees.stream()
1636            .collect(Collectors.groupingBy(
1637                Employee::getDepartment,
1638                Collectors.counting()));
1639    }
1640
1641
    // =====
    ===
1642    // Question: Average Salary By Department
1643
    // =====
    ===
1644    public static Map<String, Double> averageSalaryByDept(List<Employee>

```



```

1644 employees) {
1645
1646     return employees.stream()
1647         .collect(Collectors.groupingBy(
1648             Employee::getDepartment,
1649             Collectors.averagingDouble(Employee::getSalary)));
1650 }
1651
1652
1653 // =====
1654 // Question: Total Salary By Department
1655 // =====
1656 public static Map<String, Double> totalSalaryByDept(List<Employee>
employees) {
1657
1658     return employees.stream()
1659         .collect(Collectors.groupingBy(
1660             Employee::getDepartment,
1661             Collectors.summingDouble(Employee::getSalary)));
1662 }
1663
1664 // =====
1665 // Question: Employees Earning More Than 50000
1666 // =====
1667 public static List<Employee> highEarners(List<Employee> employees) {
1668
1669     return employees.stream()
1670         .filter(e -> e.getSalary() > 50000)
1671         .collect(Collectors.toList());
1672 }
1673
1674 // =====
1675 // Question: Sort Employees By Salary Descending
1676 // =====
1677 public static List<Employee> sortBySalaryDesc(List<Employee> employees
) {
1678
1679     return employees.stream()
1680         .sorted(Comparator.comparing(Employee::getSalary).reversed
())
1681         .collect(Collectors.toList());
1682 }
1683
1684 // =====
1685 // =====

```

```

1684    // Question: Sort By Department Then Salary
1685
1686    // =====
1687    ===
1688    public static List<Employee> sortDeptSalary(List<Employee> employees) {
1689
1690        return employees.stream()
1691            .sorted(Comparator.comparing(Employee::getDepartment)
1692                .thenComparing(Employee::getSalary))
1693            .collect(Collectors.toList());
1694    }
1695
1696    // =====
1697    ===
1698    // Question: Get Distinct Departments
1699
1700    // =====
1701    ===
1702    public static List<String> departments(List<Employee> employees) {
1703
1704        return employees.stream()
1705            .map(Employee::getDepartment)
1706            .distinct()
1707            .collect(Collectors.toList());
1708    }
1709
1710    // =====
1711    ===
1712    // Question: Find Employees In IT Department
1713
1714    // =====
1715    ===
1716    public static List<Employee> itEmployees(List<Employee> employees) {
1717
1718        return employees.stream()
1719            .filter(e -> "IT".equals(e.getDepartment()))
1720            .collect(Collectors.toList());
1721    }
1722
1723    // =====
1724    ===
1725    // Question: Group Employee Names By Department
1726
1727    // =====
1728    ===
1729    public static Map<String, List<String>> namesByDept(List<Employee>
employees) {
1730
1731        return employees.stream()
1732            .collect(Collectors.groupingBy(
1733                Employee::getDepartment,
1734                Collectors.mapping(Employee::getName, Collectors.
toList())));
1735    }

```



```

1725
1726    // =====
    ===
1727    // Question: Highest Salary Employee Department-Wise
1728
1729    // =====
    ===
1729    public static Map<String, Optional<Employee>>
highestSalaryDepartmentWise(List<Employee> employees) {
1730
1731        return employees.stream()
1732            .collect(Collectors.groupingBy(
1733                Employee::getDepartment,
1734                Collectors.maxBy(
1735                    Comparator.comparing(Employee::getSalary)
1736                )
1737            ));
1738    }
1739
1740
1741    // =====
    ===
1741    // Question: Employees Whose Name Starts With "A"
1742
1743    // =====
    ===
1743    public static List<Employee> nameStartsWithA(List<Employee> employees) {
1744
1745        return employees.stream()
1746            .filter(e -> e.getName().startsWith("A"))
1747            .collect(Collectors.toList());
1748    }
1749
1750
1751    // =====
    ===
1751    // Question: Highest Paid Employee Name Only
1752
1753    // =====
    ===
1753    public static String highestPaidEmployeeName(List<Employee> employees) {
1754
1755        return employees.stream()
1756            .max(Comparator.comparing(Employee::getSalary))
1757            .map(Employee::getName)
1758            .orElse("");
1759    }
1760
1761
1762    // =====
    ===
1762    // Question: Partition Employees By Salary > 50000
1763
1764    // =====
    ===
1764    public static Map<Boolean, List<Employee>> partitionEmployees(List<

```

```

1764 Employee> employees) {
1765
1766     return employees.stream()
1767         .collect(Collectors.partitioningBy(e -> e.getSalary() >
1768         50000));
1769     }
1770
1771     // =====
1772     // Question: Find Duplicate Employee Names
1773     // =====
1774     public static Set<String> duplicateNames(List<Employee> employees) {
1775         Set<String> seen = new HashSet<>();
1776
1777         return employees.stream()
1778             .map(Employee::getName)
1779             .filter(name -> !seen.add(name))
1780             .collect(Collectors.toSet());
1781     }
1782
1783     // =====
1784     // Question: Find Nth Highest Salary Employee
1785     // Input : employees list, n = 2
1786     // Output: Employee with 2nd highest salary
1787     // =====
1788     public static Optional<Employee> nthHighestSalary(List<Employee>
1789     employees, int n) {
1790         return employees.stream()
1791             .sorted(Comparator.comparing(Employee::getSalary).reversed
1792             ())
1793             .skip(n - 1)
1794             .findFirst();
1795     }
1796
1797     // =====
1798     // Question: LRU Cache (Least Recently Used Cache)
1799     // Uses LinkedHashMap with access-order = true
1800     // Automatically evicts the least recently used entry when capacity is
1801     // exceeded
1802     // =====
1803     public static class LRUCache<K, V> extends LinkedHashMap<K, V> {
1804         private final int capacity;

```

```

1805     public LRUCache(int capacity) {
1806         super(capacity, 0.75f, true); // true = access-order
1807         this.capacity = capacity;
1808     }
1809
1810     @Override
1811     protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
1812         return size() > capacity;
1813     }
1814 }
1815
1816
1817 // =====
1818 //
1819 // =====
1820
1821 public static void run() {
1822     System.out.println();
1823     System.out.println(
1824         "=====");
1825     System.out.println("                ARRAY CODING QUESTIONS - ALL METHODS
1826         ");
1827     System.out.println(
1828         "=====");
1829
1830     int[] arr = {10, 20, 30, 40, 50};
1831
1832 // -----
1833     System.out.println("\n1. Find Largest Element");
1834     System.out.println("    Input  : " + Arrays.toString(arr));
1835     System.out.println("    Output : " + findLargest(arr));
1836
1837 // -----
1838     System.out.println("\n2. Find Smallest Element");
1839     System.out.println("    Input  : " + Arrays.toString(arr));
1840     System.out.println("    Output : " + findSmallest(arr));
1841
1842 // -----
1843     System.out.println("\n3. Find Second Largest Element");
1844     System.out.println("    Input  : " + Arrays.toString(arr));
1845     System.out.println("    Output : " + findSecondLargest(arr));
1846
1847 // -----
1848     System.out.println("\n4. Find Second Smallest Element");
1849     System.out.println("    Input  : " + Arrays.toString(arr));
1850     System.out.println("    Output : " + findSecondSmallest(arr));
1851
1852 // -----
1853     int[] reverseArr = {1, 2, 3, 4, 5};

```

```

1850         reverseArray(reverseArr);
1851         System.out.println("\n5.  Reverse Array");
1852         System.out.println("    Input  : [1, 2, 3, 4, 5]");
1853         System.out.println("    Output : " + Arrays.toString(reverseArr));
1854
1855
// -----
1856         System.out.println("\n6.  Sum Of Array Elements");
1857         System.out.println("    Input  : " + Arrays.toString(arr));
1858         System.out.println("    Output : " + sumOfElements(arr));
1859
1860
// -----
1861         System.out.println("\n7.  Average Of Array Elements");
1862         System.out.println("    Input  : " + Arrays.toString(arr));
1863         System.out.println("    Output : " + averageOfElements(arr));
1864
1865
// -----
1866         System.out.println("\n8.  Count Even And Odd Numbers");
1867         System.out.println("    Input  : [1, 2, 3, 4, 5, 6]");
1868         countEvenOdd(new int[]{1, 2, 3, 4, 5, 6});
1869
1870
// -----
1871         System.out.println("\n9.  Find Maximum And Minimum");
1872         System.out.println("    Input  : " + Arrays.toString(arr));
1873         int[] maxMin = findMaxAndMin(arr);
1874         System.out.println("    Max    : " + maxMin[0]);
1875         System.out.println("    Min    : " + maxMin[1]);
1876
1877
// -----
1878         System.out.println("\n10. Search Element In Array");
1879         System.out.println("    Input  : " + Arrays.toString(arr) + ",
target = 30");
1880         System.out.println("    Output : " + searchElement(arr, 30));
1881
1882
// -----
1883         int[] dupArr = {1, 2, 2, 3, 4, 4, 5};
1884         System.out.println("\n11. Remove Duplicates From Array");
1885         System.out.println("    Input  : " + Arrays.toString(dupArr));
1886         System.out.println("    Output : " + Arrays.toString(
removeDuplicates(dupArr)));
1887
1888
// -----
1889         System.out.println("\n12. Find Duplicate Elements");
1890         System.out.println("    Input  : " + Arrays.toString(dupArr));
1891         System.out.println("    Output : " + findDuplicates(dupArr));
1892
1893
// -----
1894         int[] freqArr = {1, 2, 2, 3, 3, 3};
1895         System.out.println("\n13. Frequency Of Elements");
1896         System.out.println("    Input  : " + Arrays.toString(freqArr));

```

```

1897         System.out.println("    Output : " + frequencyCount(freqArr));
1898
1899
1900         // -----
1901         int[] missingArr = {1, 2, 4, 7, 10};
1902         System.out.println("\n14. Find Missing Numbers In Array");
1903         System.out.println("    Input : " + Arrays.toString(missingArr));
1904         System.out.println("    Output : " + findAllMissingNumbers(
missingArr));
1905
1906         // -----
1907         int[] zeros = {0, 1, 0, 3, 12};
1908         moveZerosToEnd(zeros);
1909         System.out.println("\n15. Move Zeros To End");
1910         System.out.println("    Input : [0, 1, 0, 3, 12]");
1911         System.out.println("    Output : " + Arrays.toString(zeros));
1912
1913         // -----
1914         System.out.println("\n18. Merge Two Arrays");
1915         System.out.println("    Input : [1, 2, 3] and [4, 5, 6]");
1916         System.out.println("    Output : " + Arrays.toString(
mergeArrays(new int[]{1, 2, 3}, new int[]{4, 5, 6})));
1917
1918         // -----
1919         System.out.println("\n19. Intersection Of Arrays");
1920         System.out.println("    Input : [1, 2, 3, 4, 5] and [4, 5, 6, 7]");
1921         System.out.println("    Output : " + Arrays.toString(
intersection(new int[]{1, 2, 3, 4, 5}, new int[]{4, 5, 6, 7
})));
1922
1923
1924         // -----
1925         System.out.println("\n20. Union Of Arrays");
1926         System.out.println("    Input : [1, 2, 3, 4, 5] and [4, 5, 6, 7]");
1927         System.out.println("    Output : " + Arrays.toString(
union(new int[]{1, 2, 3, 4, 5}, new int[]{4, 5, 6, 7})));
1928
1929
1930         // -----
1931         System.out.println("\n21. Two Sum Problem (Return Indices)");
1932         System.out.println("    Input : [2, 7, 11, 15], target = 9");
1933         List<int[]> twoSumPairs = allPairs(new int[]{2, 7, 11, 15}, 9);
1934         List<String> twoSumStr = new ArrayList<>();
1935         for (int[] p : twoSumPairs) twoSumStr.add(Arrays.toString(p));
1936         System.out.println("    Output : " + twoSumStr + " (indices)");
1937
1938         // -----
1939         System.out.println("\n22. Pair With Given Sum (Return Values)");
1940         System.out.println("    Input : [1, 2, 3, 4, 5], target = 7");
1941         List<int[]> pairValues = findPairsWithSum(new int[]{1, 2, 3, 4, 5},
7);
1942         List<String> pairStr = new ArrayList<>();
1943         for (int[] p : pairValues) pairStr.add(Arrays.toString(p));

```

```

1944         System.out.println("    Output : " + pairStr);
1945
1946
1947 // -----
1948         System.out.println("\n23. Maximum Subarray Sum (Kadane's Algorithm)"
1949 );
1950         int[] arr1 = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
1951         System.out.println("    Input : [-2, 1, -3, 4, -1, 2, 1, -5, 4]");
1952         maxSubArray(arr1);
1953         System.out.println("    (Best subarray = [4, -1, 2, 1])");
1954
1955 // -----
1956         System.out.println("\n24. Leaders In Array");
1957         System.out.println("    Input : [16, 17, 4, 3, 5, 2]");
1958         System.out.println("    Output : " + findLeaders(
1959             new int[]{16, 17, 4, 3, 5, 2}));
1960
1961 // -----
1962         int[] dnf = {2, 0, 2, 1, 1, 0};
1963         sortZeroOneTwo(dnf);
1964         System.out.println("\n26. Sort 0s 1s 2s (Dutch National Flag)");
1965         System.out.println("    Input : [2, 0, 2, 1, 1, 0]");
1966         System.out.println("    Output : " + Arrays.toString(dnf));
1967
1968 // -----
1969         System.out.println("\n27. Product Of Array Except Self");
1970         System.out.println("    Input : [1, 2, 3, 4]");
1971         System.out.println("    Output : " + Arrays.toString(
1972             productExceptSelf(new int[]{1, 2, 3, 4})));
1973
1974 // -----
1975         System.out.println("\n28. Best Time To Buy And Sell Stock");
1976         System.out.println("    Input : [7, 1, 5, 3, 6, 4]");
1977         maxProfit(new int[]{7, 1, 5, 3, 6, 4});
1978
1979 // -----
1980         System.out.println("\n29. First Non-Repeating Element");
1981         System.out.println("    Input : [9, 4, 9, 6, 7, 4]");
1982         System.out.println("    Output : " + firstNonRepeating(
1983             new int[]{9, 4, 9, 6, 7, 4}));
1984
1985 // -----
1986         System.out.println("\n30. Contains Duplicate");
1987         System.out.println("    Input : [1, 2, 3, 1]");
1988         System.out.println("    Output : " + containsDuplicate(
1989             new int[]{1, 2, 3, 1}));
1990
1991 // -----

```



```

1992      System.out.println("\n31. Sort Array Ascending (Stream)");
1993      System.out.println("    Input  : [5, 3, 8, 1, 2]");
1994      System.out.println("    Output : " + Arrays.toString(
1995          sortAscendingStream(new int[]{5, 3, 8, 1, 2})));
1996
1997
// -----
1998      System.out.println("\n32. Sort Array Descending (Stream)");
1999      System.out.println("    Input  : [5, 3, 8, 1, 2]");
2000      System.out.println("    Output : " + Arrays.toString(
2001          sortDescendingStream(new int[]{5, 3, 8, 1, 2})));
2002
2003
// -----
2004      System.out.println("\n33. Separate Odd and Even Numbers (Stream)");
2005      System.out.println("    Input  : [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]");
2006      findOddEvenNumbers(new int[]{1, 2, 3, 4, 5, 6, 7, 8, 9, 10});
2007
2008
// -----
2009      System.out.println("\n34. Next Greater Element");
2010      System.out.println("    Input  : [4, 5, 2, 10, 8]");
2011      System.out.println("    Output : " + Arrays.toString(
2012          nextGreaterElement(new int[]{4, 5, 2, 10, 8})));
2013
2014
// -----
2015      int[] rotateArr = {1, 2, 3, 4, 5, 6, 7};
2016      rotateArrayByK(rotateArr, 3);
2017      System.out.println("\n35. Rotate Array By K = 3 (Right Rotation)");
2018      System.out.println("    Input  : [1, 2, 3, 4, 5, 6, 7]");
2019      System.out.println("    Output : " + Arrays.toString(rotateArr));
2020
2021
// -----
2022      System.out.println("\n36. Find Missing Single Number (1 to N)");
2023      System.out.println("    Input  : [1, 2, 4, 5, 6], n = 6");
2024      System.out.println("    Output : " + findSingleMissingNumber(
2025          new int[]{1, 2, 4, 5, 6}, 6));
2026
2027
// -----
2028      List<List<Integer>> nested = Arrays.asList(
2029          Arrays.asList(1, 2),
2030          Arrays.asList(3, 4, 5));
2031      System.out.println("\n37. Flatten Nested List");
2032      System.out.println("    Input  : [[1, 2], [3, 4, 5]]");
2033      System.out.println("    Output : " + flattenList(nested));
2034
// -----
2035      List<Employee> emps = Arrays.asList(
2036          new Employee("Alice", "IT", 70000),
2037          new Employee("Bob", "HR", 50000),
2038          new Employee("Charlie", "IT", 90000),
2039          new Employee("David", "Finance", 60000),
2040          new Employee("Eve", "HR", 75000)
2041      );

```

```

2042
2043     System.out.println("\n38. Group Employees By Department");
2044     System.out.println("    Output : " + groupEmployeesByDept(emps));
2045
2046     System.out.println("\n39. Highest Salary Employee By Department");
2047     System.out.println("    Output : " + highestSalaryByDept(emps));
2048
2049 //     System.out.println("\n40. Employee With Highest Salary");
2050 //     System.out.println("    Output : " + highestSalary(emps).orElse(
2051 //         null));
2052 //     System.out.println("\n41. Second Highest Salary Employee");
2053 //     System.out.println("    Output : " + secondHighestSalary(emps).
2054 //         orElse(null));
2055
2056     System.out.println("\n42. Top 3 Highest Paid Employees");
2057     System.out.println("    Output : " + top3Employees(emps));
2058
2059     System.out.println("\n43. Count Employees In Each Department");
2060     System.out.println("    Output : " + countByDepartment(emps));
2061
2062     System.out.println("\n44. Average Salary By Department");
2063     System.out.println("    Output : " + averageSalaryByDept(emps));
2064
2065     System.out.println("\n45. Total Salary By Department");
2066     System.out.println("    Output : " + totalSalaryByDept(emps));
2067
2068     System.out.println("\n46. Employees Earning More Than 50000");
2069     System.out.println("    Output : " + highEarners(emps));
2070
2071     System.out.println("\n47. Sort Employees By Salary Descending");
2072     System.out.println("    Output : " + sortBySalaryDesc(emps));
2073
2074     System.out.println("\n48. Distinct Departments");
2075     System.out.println("    Output : " + departments(emps));
2076
2077     System.out.println("\n49. Employees In IT Department");
2078     System.out.println("    Output : " + itEmployees(emps));
2079
2080     System.out.println("\n50. Nth Highest Salary (n=2)");
2081     System.out.println("    Output : " + nthHighestSalary(emps, 2).
2082     orElse(null));
2083
2084     System.out.println("\n53. Subset Sum Combinations");
2085     System.out.println("    Input : nums = [2, 7, 3, 6, 4, 5], target
2086     = 9");
2087     SubsetSum.findCombinations(new int[]{2, 7, 3, 6, 4, 5}, 9);
2088
2089     System.out.println("\n
2090     =====");
2091     System.out.println("                ALL METHODS EXECUTED
2092     !                ");
2093     System.out.println(
2094     "=====");
2095     System.out.println();
2096 }
2097

```



```
2092 }
2093
```